

Recorrido de matrices

Programación II

Arreglos

Un arreglo es una secuencia de elementos con o sin elementos repetidos.

un arreglo tiene la forma $arr = [arr_0, arr_1, \dots, arr_{n-1}]$

Por ejemplo en programación, `int[] arr = { 4, 3, 5, 9, 5};`

Y la forma de consultar sus elementos en `arr[índice]`

- `arr[0] == 4`
- `arr[2] == 5`
- `arr[5] == ?`

Arreglos

Un arreglo es una secuencia de elementos con o sin elementos repetidos.

un arreglo tiene la forma $arr = [arr_0, arr_1, \dots, arr_{n-1}]$

Por ejemplo en programación, `int[] arr = { 4, 3, 5, 9, 5};`

Y la forma de consultar sus elementos en `arr[índice]`

- `arr[0] == 4`
- `arr[2] == 5`
- `arr[5] => Lanza excepción (ArrayIndexOutOfBoundsException)`

Matrices

Una matriz es una tabla bidimensional de elementos.

Por ejemplo, las siguientes son matrices de m filas y n columnas ($m \times n$):

Matemáticas

$$A =_{m \times n} \begin{bmatrix} a_{11} & a_{12} & a_{13} & \dots & a_{1n} \\ a_{21} & & & & \\ a_{31} & & & & \\ \vdots & & & & \\ a_{m1} & & & & a_{mn} \end{bmatrix}$$

Programación

$$A =_{m \times n} \begin{bmatrix} a_{00} & a_{01} & a_{02} & \dots & a_{0(n-1)} \\ a_{10} & & & & \\ a_{20} & & & & \\ \vdots & & & & \\ a_{(m-1)0} & & & & a_{(m-1)(n-1)} \end{bmatrix}$$

Matrices - Creación

```
int CANT_FILAS=2
int CANT_COLUMNAS=3;
int[][] matriz = new int[CANT_FILAS][CANT_COLUMNAS];
matriz[0][0]=1;
matriz[0][1]=2;
matriz[0][2]=3;
matriz[1][0]=4;
matriz[1][1]=5;
matriz[1][2]=6;

int[][] matriz2 = {
    {1, 2, 3},
    {4, 5, 6}
};
```

Matrices - Recorrido por fila

```
for (int f=0; f<matriz.length; f++)  
    for (int c=0; c<matriz[0].length; c++)  
        System.out.println(matriz[f][c]);
```

Matrices - Recorrido por fila

```
for (int f=0; f<matriz.length; f++)  
    for (int c=0; c<matriz[0].length; c++)  
        System.out.println(matriz[f][c]);
```

F = 0

C = 0

Salida:

```
int[][] matriz = {  
    {1, 2, 3},  
    {4, 5, 6}  
};
```

Matrices - Recorrido por fila

```
for (int f=0; f<matriz.length; f++)  
    for (int c=0; c<matriz[0].length; c++)  
        System.out.println(matriz[f][c] + " ");
```

F = 0

C = 0

Salida: 1

```
int[][] matriz = {  
    {1, 2, 3},  
    {4, 5, 6}  
};
```

Matrices - Recorrido por fila

```
for (int f=0; f<matriz.length; f++)  
    for (int c=0; c<matriz[0].length; c++)  
        System.out.println(matriz[f][c] + " ");
```

F = 0

C = 1

Salida: 1 2

```
int[][] matriz = {  
    {1, 2, 3},  
    {4, 5, 6}  
};
```

Matrices - Recorrido por fila

```
for (int f=0; f<matriz.length; f++)  
    for (int c=0; c<matriz[0].length; c++)  
        System.out.println(matriz[f][c] + " ");
```

F = 0

C = 2

Salida: 1 2 3

```
int[][] matriz = {  
    {1, 2, 3},  
    {4, 5, 6}  
};
```

Matrices - Recorrido por fila

```
for (int f=0; f<matriz.length; f++)  
    for (int c=0; c<matriz[0].length; c++)  
        System.out.println(matriz[f][c] + " ");
```

F = 1
C = 0

Salida: 1 2 3 4

```
int[][] matriz = {  
    {1, 2, 3},  
    {4, 5, 6}  
};
```

Matrices - Recorrido por fila

```
for (int f=0; f<matriz.length; f++)  
    for (int c=0; c<matriz[0].length; c++)  
        System.out.println(matriz[f][c] + " ");
```

F = 1

C = 1

Salida: 1 2 3 4 5

```
int[][] matriz = {  
    {1, 2, 3},  
    {4, 5, 6}  
};
```

Matrices - Recorrido por fila

```
for (int f=0; f<matriz.length; f++)  
    for (int c=0; c<matriz[0].length; c++)  
        System.out.println(matriz[f][c] + " ");
```

F = 1

C = 2

Salida: 1 2 3 4 5 6

```
int[][] matriz = {  
    {1, 2, 3},  
    {4, 5, 6}  
};
```

Matrices - Recorrido por columna

```
for (int c=0; c<matriz[0].length; c++)  
    for (int f=0; f<matriz.length; f++)  
        System.out.println(matriz[f][c] + " ");
```

Matrices - Recorrido por columna

```
for (int c=0; c<matriz[0].length; c++)  
    for (int f=0; f<matriz.length; f++)  
        System.out.println(matriz[f][c] + " ");
```

F = 0

C = 0

Salida: 1

```
int[][] matriz = {  
    {1, 2, 3},  
    {4, 5, 6}  
};
```

Matrices - Recorrido por columna

```
for (int c=0; c<matriz[0].length; c++)  
    for (int f=0; f<matriz.length; f++)  
        System.out.println(matriz[f][c] + " ");
```

F = 1

C = 0

Salida: 1 4

```
int[][] matriz = {  
    {1, 2, 3},  
    {4, 5, 6}  
};
```

Matrices - Recorrido por columna

```
for (int c=0; c<matriz[0].length; c++)  
    for (int f=0; f<matriz.length; f++)  
        System.out.println(matriz[f][c] + " ");
```

F = 0

C = 1

Salida: 1 4 2

```
int[][] matriz = {  
    {1, 2, 3},  
    {4, 5, 6}  
};
```

Matrices - Recorrido por columna

```
for (int c=0; c<matriz[0].length; c++)  
    for (int f=0; f<matriz.length; f++)  
        System.out.println(matriz[f][c] + " ");
```

F = 1

C = 1

Salida: 1 4 2 5

```
int[][] matriz = {  
    {1, 2, 3},  
    {4, 5, 6}  
};
```

Matrices - Recorrido por columna

```
for (int c=0; c<matriz[0].length; c++)  
    for (int f=0; f<matriz.length; f++)  
        System.out.println(matriz[f][c] + " ");
```

F = 0

C = 2

Salida: 1 4 2 5 3

```
int[][] matriz = {  
    {1, 2, 3},  
    {4, 5, 6}  
};
```

Matrices - Recorrido por columna

```
for (int c=0; c<matriz[0].length; c++)  
    for (int f=0; f<matriz.length; f++)  
        System.out.println(matriz[f][c] + " ");
```

F = 1

C = 2

Salida: 1 4 2 5 3 6

```
int[][] matriz = {  
    {1, 2, 3},  
    {4, 5, 6}  
};
```

Matrices y acumuladores booleanos

- A) Implementar la función `boolean algunaFilaPar(int[][] mat);`
Que dada una matriz, devuelve true si la misma tiene al menos una fila con todos sus elementos pares.
Si la matriz está vacía, devuelve true.
- B) Implementar la función `boolean algunaColumnaPar(int[][] mat);`
Que dada una matriz, devuelve true si la misma tiene al menos una columna con todos sus elementos pares.
Si la matriz está vacía, devuelve true.